

S31212 - Accelerating Machine Learning Applications Using CUDA Graph and OpenACC.

Leonel Toledo (BSC), Pedro Valero-Lara(ORNL), Antonio J. Peña (BSC).



The EPEEC project has received funding from the European Union's Horizon 2020 research and innovation programme under the grant agreement N° 801051

INTRODUCTION



Introduction

- GPU compute capabilities have been significantly increasing; however, applications and scalability of algorithms still face some important challenges. One important problem regarding scalability is hardware resource assignment.
- Tasking, on the other hand makes it possible for algorithms with run-time dependent execution flow, to be parallelized.
 - For instance, take as an example a loop with independent chunks of work and some of those chunks potentially could be executed at the same time. Tasking provides a solution for this kind of scenarios



Introduction

- Machine learning is more relevant than ever, many fields benefit from its development and breakthroughs.
- By combining different programming models, developers can easily accelerate applications without compromising the performance of their code.
- There are many fields that benefit from these kind of applications, such as: High performance computing, computer graphics, AI, speech recognition, traffic prediction and many more.

RELATED WORK

Related Work

- Programming Models:
 - StarPU
 - Legion
 - Thread Building Blocks
 - High Performance ParallelX
 - OpenMP
 - OmpSs
 - For more details: E31193 - A Tale of Two Programming-Models: Enhancing Heterogeneity, Productivity, and Performance through OmpSs-2 + OpenACC Inter-Operation
 - CUDA Dynamic Parallelism
CUDA Graphs.
Linear Algebra Routines
 - CuBLAS
 - CuSparse
 - LAS

IMPLEMENTATION



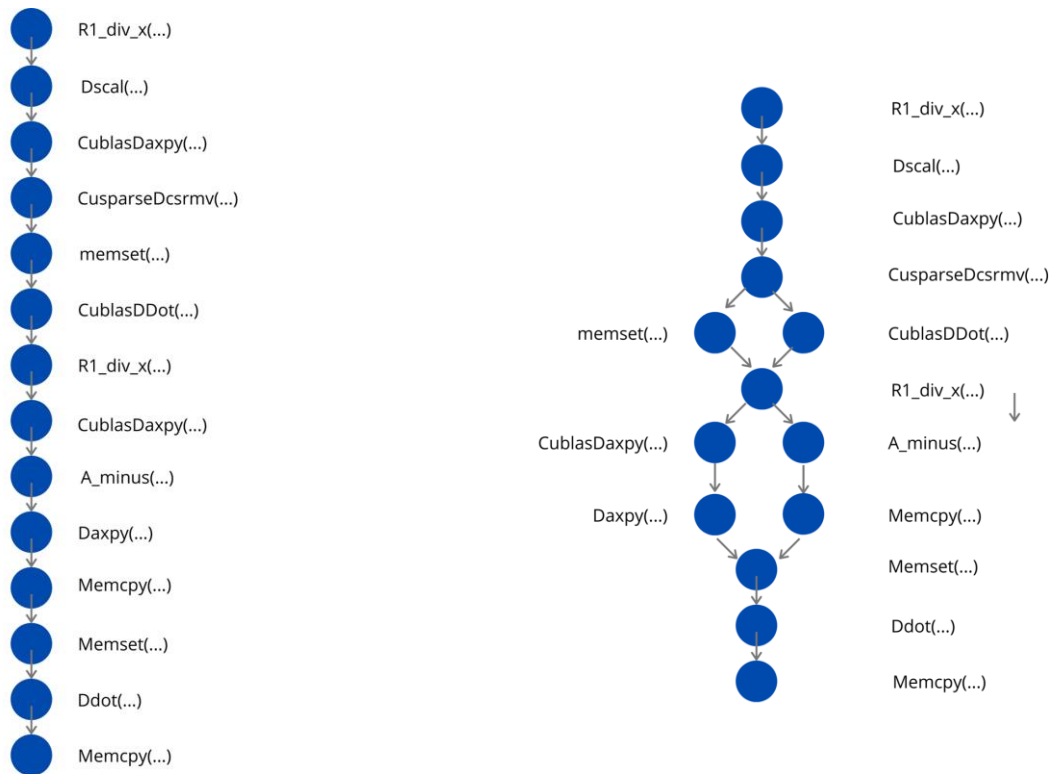
Task-Based Programming using GPU

- Alternatives for asynchronous kernel execution in GPUs
 - CUDA Streams
 - Dynamic Parallelism
 - CUDA Graphs API

CUDA Graphs

- Many HPC applications such as deep neural network training and scientific simulations have an iterative structure where the same workflow is executed repeatedly. CUDA streams require that the work be resubmitted with every iteration, which consumes both time and CPU resources. Graphs enable a *define-once-run-repeatedly* execution flow.

CUDA Graphs Algorithm Representation





OPENACC

- High-level directive-based programming model.
 - Supports C, C++ and Fortran.
- Allows Programmers to interact with heterogeneous high-performance computing architectures.
- Developers insert hints into their code and the compiler decides how to offload it to the GPU

Application (Particle Swarm Optimization)



PSO Overview

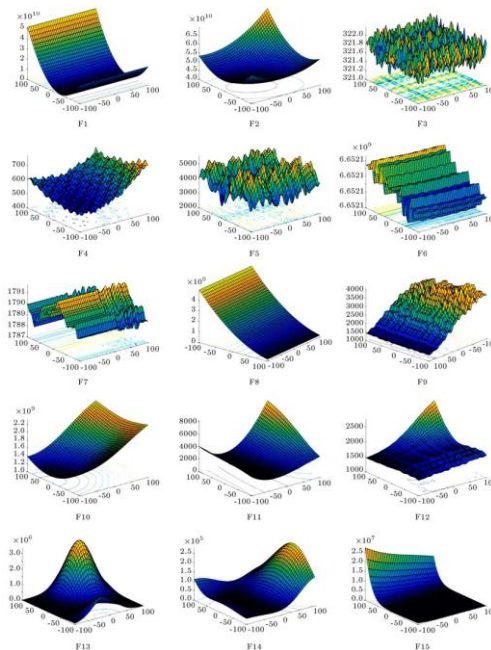
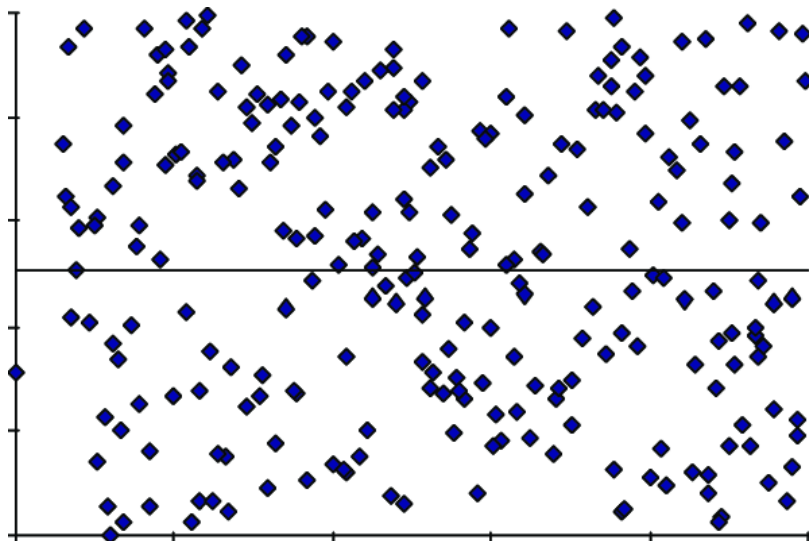
- Based on nature. (Similar to bird flocking or fish schooling)
- Population-based, stochastic optimization technique.
- Shares similarities with Genetic algorithms. (However there is no crossover or mutation operators).

PSO Overview

- System is initialized with random population within the boundaries of the environment. Each solution searches for optima.
- Iterative algorithm that updates generations until at least one particle reaches the solution.
- Applications Range from function optimization, artificial neural network training, fuzzy system control and many more.

Initialization.

PSO Initialization of the search space.

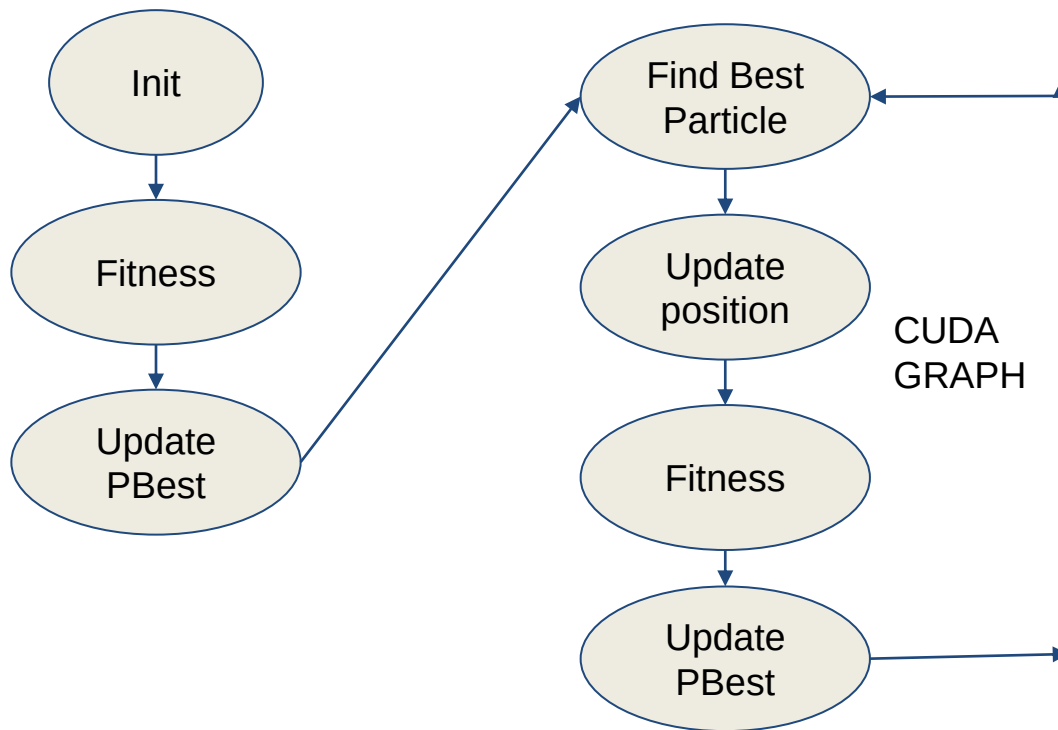


PSO Definition

- For each particle we define a set of variables where we evaluate any given function.
- The result of the evaluation determines the fitness of the particle.

```
typedef struct particle{  
  
    double posX;  
    double posY;  
    double posZ;  
  
    double speedX;  
    double speedY;  
    double speedZ;  
  
    double bestX;  
    double bestY;  
    double bestZ;  
  
    double fitness;  
    double bestValue;  
    double currentValue;  
  
} Particle;
```

Workflow



Update Task.

Each iteration every particle computes the following equations:

$$(1) \mathbf{v}[] = \mathbf{v}[] + c1 * \text{rand}() * (\mathbf{pbest}[] - \mathbf{present}[]) + c2 * \text{rand}() * (\mathbf{gbest}[] - \mathbf{present}[])$$

$$(2) \mathbf{present}[] = \mathbf{present}[] + \mathbf{v}[] \quad (b)$$

- $\mathbf{V}[]$ is a 3D vector that represents the speed of a particle in each direction within the search space (x,y,z)
- Pbest is the best value that particle(i) has achieved so far
- Present is the current position/value of a particle(i) every iteration
- gBest is the global best fitness achieved (all particles)
- C1 and C2 are constants which are used to determine the impact of each side of the equation. It's value decreases each iteration, this way at the beginning particles are bound to move larger distances (to avoid local minima and maxima in hills). Also used as learning factors.

Update.

- Equation 1 is composed of two parts.
 - **$c1 * rand() * (pbest[] - present[])$**
 - This allows particles to explore within the space taking as a reference the best solution that has been found so far.
 - **$c2 * rand() * (gbest[] - present[])$**
 - This side of the equation takes into consideration the best values found by the entire population, particles with low fitness values will tend to come closer to particles with better fitness.

Algorithm implementation

```
void fitnessTask(){  
    for(int i=0; i<POPULATION; i++){  
        calculateFitness(particleVector[i],OBJECTIVE);  
    }  
}
```

```
void updatePBestTask(){  
    for(int i=0; i<POPULATION; i++){  
        updatePBest(particleVector[i],OBJECTIVE);  
    }  
}
```

```
void findBestParticleTask(){  
    findBestParticle(particleVector,OBJECTIVE,GlobalBestFitness
```



Algorithm implementation

```
initTask();  
fitnessTask();  
updatePBestTask();  
for (int i=0; i<ITERATIONS; i++){  
    findBestParticleTask();  
    updateSpeedVectorTask();  
    fitnessTask();  
    updatePBestTask();  
}
```

Implementation OpenACC

```
//Sample Kernel
void fitnessTask(){
#pragma acc kernels deviceptr(partArray) async(0)
    for(int i=0; i<POPULATION; i++){
        .
        .
        .    //Individual calculations for each element in the population
        .
        .
    }
}
```

Implementation CUDA Graphs

```
//RECORD FIRST ITERATION
```

```
    cudaStreamBeginCapture(stream1 , cudaStreamCaptureModeGlobal);  
    findBestParticleTask();  
    updateSpeedVectorTask();  
    fitnessTask();  
    updatePBestTask();  
    cudaStreamEndCapture(stream1 , &graph);  
    cudaGraphExec_t graphExec;  
    cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);  
    for (int i=0; i<ITERATIONS; i++){  
        cudaGraphLaunch(graphExec, streamForGraph);  
    }
```

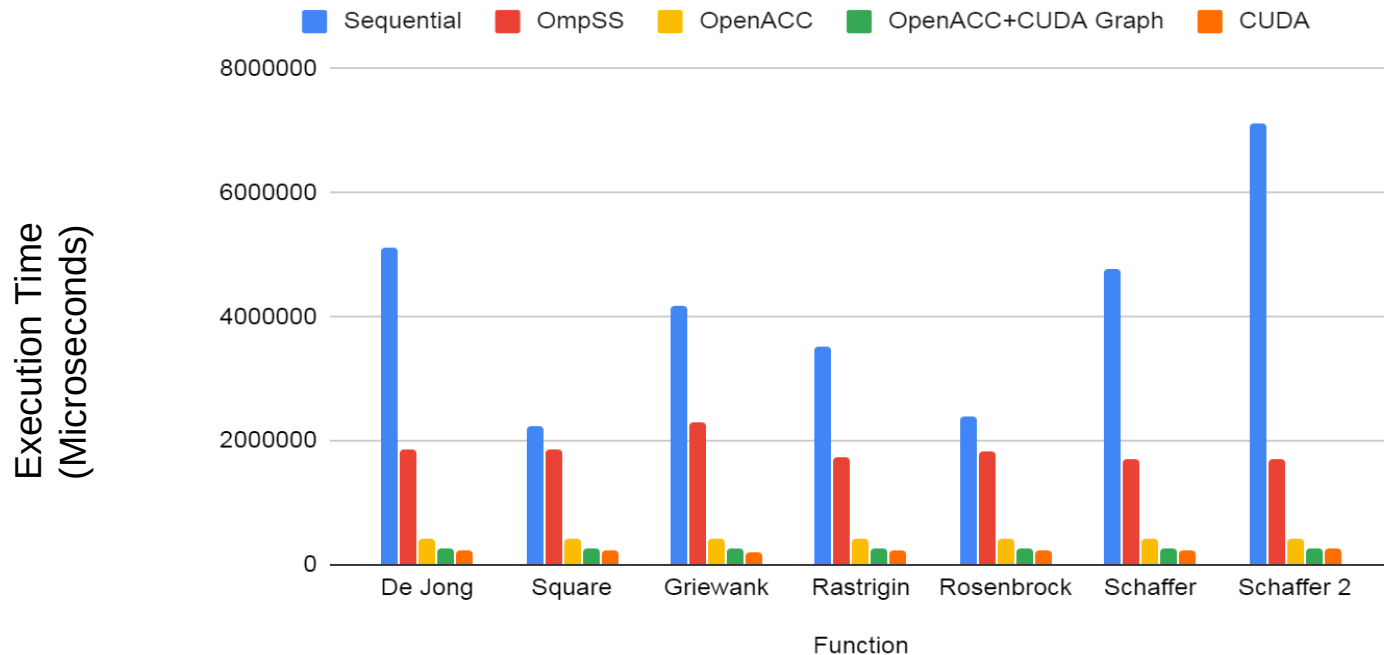

Evaluation

Functions used for evaluation of the algorithm.

Function	Formula
Sphere	$f(x) = \sum_{i=0}^n x_i^2$
De Jong	$f(x) = \sum_{i=0}^n i * x_i^4$
Griewank	$f(x) = \frac{1}{4000} \sum_{i=0}^n x_i^2 - \prod_{i=0}^n \cos(\frac{x_i}{\sqrt{i}}) + 1$
Rastrigin	$f(x) = \sum_{i=0}^n [x_i^2 - 10 \cos(2\pi * x_i) + 10]$
Rosenbrock	$f(x) = \sum_{i=0}^n [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$
Schaffer	$f(x) = \sum_{i=0}^{n-1} [0.5 + \frac{(\sin \sqrt{x_i^2 + x_{i+1}^2}) - 0.5}{(1 + .001(x_i^2 + x_{i+1}^2))^2}]$
Schaffer 2	$f(x) = (\sum_{i=0}^n x_i^2)^{.25} [1 + (50(\sum_{i=0}^n x_i^2)^{0.1})^2]$

We carry out the performance evaluation by using the following heterogeneous system: 2 x IBM Power9 8335-GTH at 2.4 Ghz, 32 GB RAM memory, and a NVIDIA V100 (Volta) GPU with 16GB HBM2 and NVLink2 for high-bandwidth communication between CPU and GPU

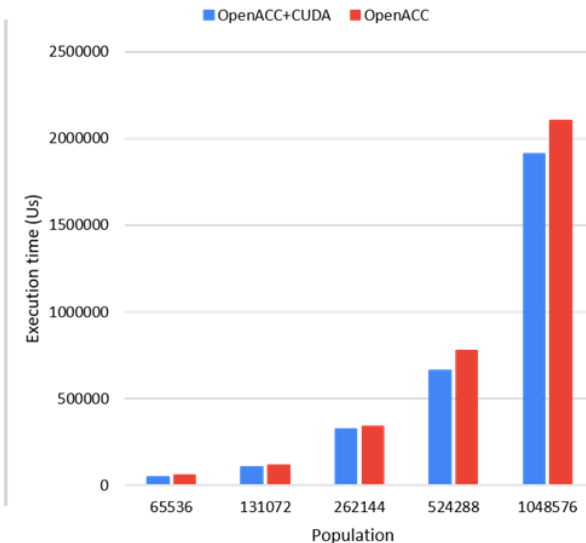
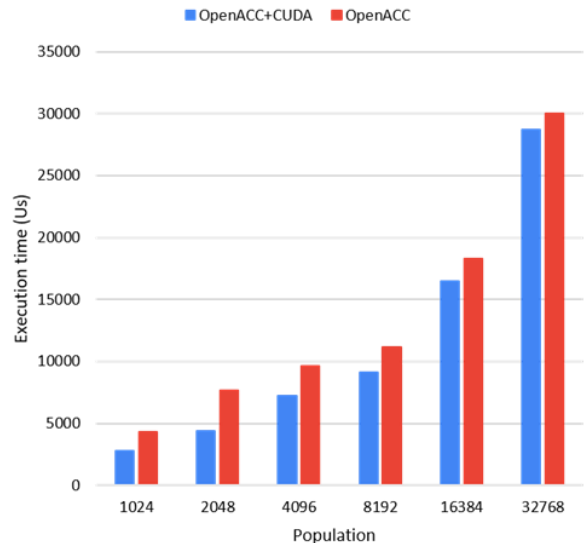
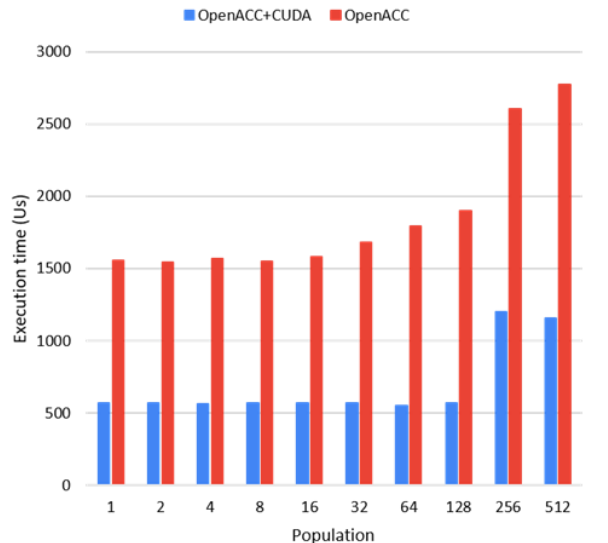
Results



Multicore version is at least 3x faster than Sequential.

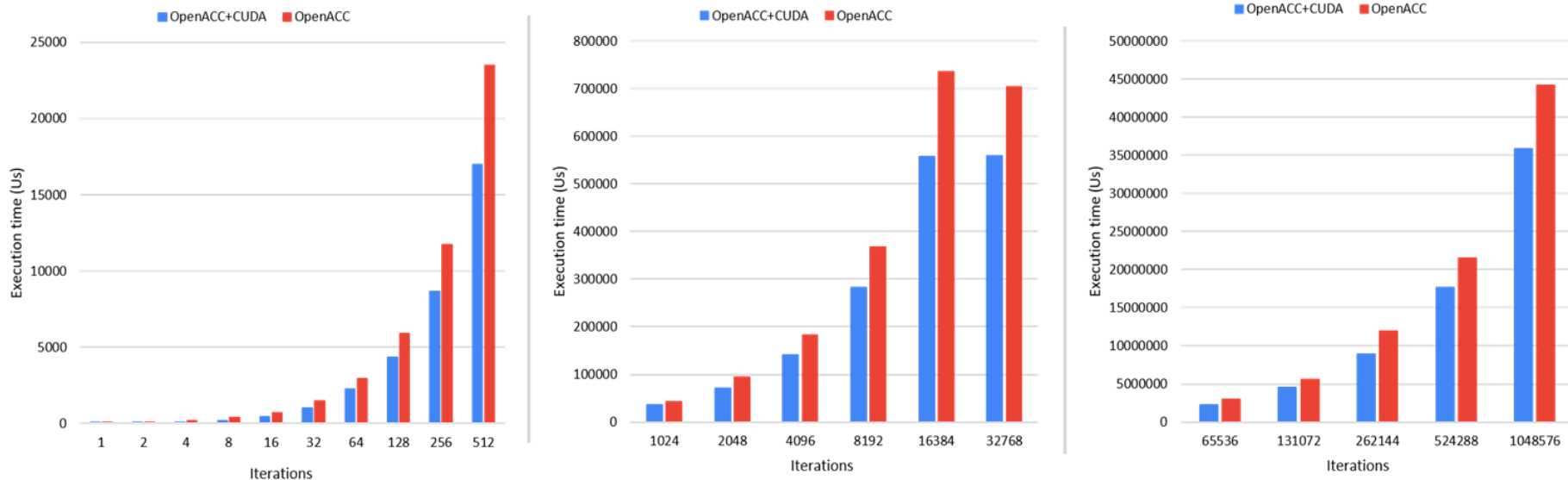
Hardware Accelerated versions are at least 8x faster.

Population Performance



Execution time comparison between OpenACC+Graphs and pure OpenACC (lower is better)

Iteration Performance



Execution time comparison between OpenACC+Graphs and pure OpenACC (lower is better)

Programming Model Integration

Syntax proposal

```
//Iterative Phase
```

```
for (int i=0; i<ITERATIONS; i++){  
    smpTask1();  
    #pragma oss taskGraph  
    findBestParticleTask(x);  
    updateSpeedVectorTask(x);  
    fitnessTask(x,y);  
    updatePBestTask(x);  
    #pragma oss endGraph  
    smpTask2();  
}
```

Considerations:

- Developer must know that variables (i.e x, y) should reside in GPU.
- Boilerplate is always the same for the creation/execution of the Graph so it could be assumed and generated by the compiler when encountering the graph pragmas.

Conclusions

- Task-based programming model that can efficiently be used on GPUs no longer limiting execution to a single Kernel.
- Shows performance improvements while maintaining simplicity. Enhancing coding productivity and scalability.
- We provide a mechanism to easily leverage GPU and offload parallel code into the GPU without major efforts.

Thank You!



www.epeec-project.eu



The EPEEC project has received funding from the European Union's Horizon 2020 research and innovation programme under the grant agreement N° 801051